LECTURE 01

HTML & CSS,

the page.

Structure, style, layout, and the first mental model for every web page you'll build.

tags | semantics | selectors | box model | flexbox | responsive

STRUCTURE

HTML says what is on the page.

HTML is HyperText Markup Language.

It defines structure and content: headings, paragraphs, images, buttons, forms, sections.

It does not decide the final visual design. That is CSS.

HTML is the noun layer.

```
<h1>Hello, World!</h1>
<p>This is a paragraph.</p>
```

SKELETON

Every page starts with the same bones.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>My Page</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <h1>Hello!</h1>
    <p>Content goes here.</p>
  </body>
</html>
```

HEAD VS BODY

Metadata up top. Visible stuff below.

DOCTYPE

Tells the browser this is modern HTML.

HEAD

Metadata: title, charset, viewport, CSS links.

BODY

Everything the user actually sees on the page.

CHARSET

Character encoding. Use UTF-8.

VIEWPORT

Makes layout behave on mobile devices.

LINK

Connects your HTML file to external CSS.

ATTRIBUTES

Tags can carry extra information.

```
<a href="https://google.com" target="_blank">
  Go to Google
</a>

<img src="photo.jpg" alt="A photo of a dog" />

<input type="text" placeholder="Enter your name" />
```

- ✓ `href` - where a link goes
- ✓ `src` - source file
- ✓ `alt` - image description
- ✓ `class` - CSS/JS grouping
- ✓ `id` - unique identifier
- ✓ `type` - input kind

COMMON TAGS

Most pages are built from a small vocabulary.

```html
<!-- Text -->
<h1>Largest heading</h1>
<h2>Second level</h2>
<p>A paragraph of text.</p>
<strong>Bold</strong>
<em>Italic</em>
<span>Inline container</span>

<!-- Links and images -->
<a href="/about">About</a>
<img src="dog.jpg" alt="A dog" />
```

```html
<!-- Lists -->
<ul>
  <li>Apples</li>
  <li>Bananas</li>
</ul>

<ol>
  <li>Wake up</li>
  <li>Write code</li>
</ol>

<!-- Containers -->
<div>Block container</div>
<span>Inline container</span>
```

one h1 per page. use heading levels for structure, not font size.

SEMANTIC HTML

Use tags that say what they mean.

<header> <nav> <main> <section> <footer>

Semantics help screen readers, search engines, and teammates understand the page.

SEMANTIC CONTRAST

Not everything should be a div.

LESS MEANINGFUL

```
<div class="header">
  <div class="nav">...</div>
</div>
```

MORE MEANINGFUL

```
<header>
  <nav>...</nav>
</header>
```

div is a box. semantic tags are boxes with a job title.

FORMS

Forms collect user input.

```html
<form action="/submit" method="POST">
  <label for="username">Username</label>
  <input type="text" id="username" name="username" required />

  <label for="email">Email</label>
  <input type="email" id="email" name="email" />

  <textarea id="bio" name="bio"></textarea>
  <button type="submit">Submit</button>
</form>
```

- ✓ `label` connects to input via `for` + `id`.
- ✓ `name` identifies submitted data.
- ✓ `required` lets the browser block empty submission.
- ✓ Later: JavaScript can intercept submit with `preventDefault()`.

LAYOUT BEHAVIOR

Block stacks. Inline flows.

BLOCK

Full width, new line.

Examples: `div`, `p`, headings, lists, forms.

INLINE

Only content width.

Examples: `span`, `a`, `strong`, `em`, `img`.

this is why paragraphs stack and links sit inside sentences.

STYLE LAYER

CSS says how it looks.

Colors, fonts, spacing, layout, animation, and responsiveness all live here.

CSS is the adjective and layout layer.

WHERE CSS LIVES

External first. Inline last.

PREFERRED

```
<link rel="stylesheet" href="styles.css" />
```

OKAY FOR DEMOS

```
<style>
  h1 { color: navy; }
</style>
```

AVOID

```
<h1 style="color: navy;">
  Hello
</h1>
```

reusable styles belong in reusable files.

RULE ANATOMY

Selector. Property. Value.

```
selector {
  property: value;
  property: value;
}

h1 {
  color: navy;
  font-size: 32px;
  font-weight: bold;
}
```

- ✓ Selector: which elements?
- ✓ Property: what changes?
- ✓ Value: what does it become?
- ✓ Declarations end with semicolons.

SELECTORS

Target the right elements.

```css
/* Element */
p { color: gray; }

/* Class */
.card { background: white; }

/* ID */
#header { background: navy; }

/* Descendant */
.card p { font-size: 14px; }
```

```css
/* Direct child */
.card > h2 { margin: 0; }

/* Multiple */
h1, h2, h3 { font-family: sans-serif; }

/* Pseudo-class */
a:hover { color: red; }
button:disabled { opacity: 0.5; }

/* Pseudo-element */
p::first-letter { font-size: 2em; }
```

CASCADE

Which rule wins?

SPECIFICITY LADDER

- ✓ `!important` overrides everything. Avoid it.
- ✓ Inline style is highest.
- ✓ ID selectors are high.
- ✓ Classes and pseudo-classes are medium.
- ✓ Elements are low.

```
p { color: gray; }          /* loses */
.intro { color: blue; }     /* wins over element */
#main p { color: green; }   /* wins over class */

/* If specificity ties, later wins. */
```

BOX MODEL

Every element is a rectangle.

Content is the thing. Padding is inside space. Border is the outline. Margin is outside space.

SIZING SANITY

border-box saves headaches.

```css
.box {
  width: 300px;
  height: 150px;
  padding: 20px;
  border: 2px solid black;
  margin: 10px;
}
```

```css
/* Put this at the top */
*, *::before, *::after {
  box-sizing: border-box;
}

/* shorthand */
padding: 20px;
padding: 10px 20px;
padding: 10px 20px 10px 20px;
```

without border-box, padding and border make boxes bigger than expected.

DISPLAY

Display controls layout behavior.

```
display: block;        /* full width */
display: inline;       /* flows with text */
display: inline-block; /* inline + width/height */
display: none;         /* removed entirely */
display: flex;         /* flex layout container */
```

This property decides how the element participates in layout.

`display: flex` is not magic on the children. You apply it to the parent container.

display chooses the layout game.

FLEXBOX

Flexbox arranges children.

```
.container {
  display: flex;
  flex-direction: row;
  justify-content: center;
  align-items: center;
  gap: 16px;
  flex-wrap: wrap;
}
```

- ✓ `flex-direction` chooses row or column.
- ✓ `justify-content` aligns on the main axis.
- ✓ `align-items` aligns on the cross axis.
- ✓ `gap` creates clean space between children.

FLEX CHILDREN

Children can grow, shrink, and override.

PARENT VALUES

- ✓ flex-start
- ✓ flex-end
- ✓ center
- ✓ space-between
- ✓ space-around
- ✓ space-evenly

```css
.child {
  flex: 1;
  flex-grow: 2;
  flex-shrink: 0;
  align-self: flex-end;
}
```

parent controls the group. child rules tweak one item.

POSITIONING + VISUAL SYSTEM

Flow first.
Offsets when needed.

```css
position: static;    /* default */
position: relative;  /* offset but still in flow */
position: absolute;  /* removed from flow */
position: fixed;     /* fixed to viewport */
position: sticky;    /* sticks after threshold */

.tooltip {
  position: absolute;
  top: 0;
  right: 0;
}
```

```css
/* Color */
color: red;
color: #ff5733;
background-color: rgba(0, 0, 0, 0.5);
color: hsl(12, 100%, 60%);

/* Type */
font-family: "Inter", Arial, sans-serif;
font-size: 1rem;
font-weight: 700;
line-height: 1.5;
```

RESPONSIVE + REUSABLE

Write defaults. Enhance upward.

```css
/* Variables */
:root {
  --primary-color: #4f46e5;
  --spacing-md: 16px;
  --border-radius: 8px;
}

.button {
  background-color: var(--primary-color);
  padding: var(--spacing-md);
}
```

```css
/* Default: mobile */
.container {
  flex-direction: column;
  padding: 16px;
}

/* Tablet and up */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
    padding: 32px;
  }
}
```

mobile-first: small screen styles first, then min-width media queries.